

Sri Lanka Institute of Information Technology



IT3091 – Machine Learning

Year 2, Semester 2- 2026

Practical Answer Submission

Practical Sheet No: 04

Student ID	IT24103352
Student Name	M.A.W.AHAMED
Campus/ Center name	Kandy Uni
Specialization	AI

Loading Dataset

In [77]:

```
# Install required package if needed
# !pip install imbalanced-learn

import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.feature_selection import SequentialFeatureSelector
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

from imblearn.over_sampling import RandomOverSampler, SMOTE
from imblearn.under_sampling import RandomUnderSampler
```

In [78]:

```
df = sns.load_dataset('titanic')

df.head()
```

Out[78]:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	embark_town	alive	alone
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN	Southampton	no	False
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C	Cherbourg	yes	False
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN	Southampton	yes	True
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C	Southampton	yes	False
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN	Southampton	no	True

In [79]:

```
print(df.shape)
print(df.columns)
print(df.info())
```

(891, 15)

Index(['survived', 'pclass', 'sex', 'age', 'sibsp', 'parch', 'fare',
 'embarked', 'class', 'who', 'adult_male', 'deck', 'embark_town',
 'alive', 'alone'],
 dtype='object')

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 891 entries, 0 to 890

Data columns (total 15 columns):

#	Column	Non-Null Count	Dtype
0	survived	891 non-null	int64
1	pclass	891 non-null	int64
2	sex	891 non-null	object
3	age	714 non-null	float64
4	sibsp	891 non-null	int64
5	parch	891 non-null	int64
6	fare	891 non-null	float64

```

6   fare            891 non-null    float64
7   embarked       889 non-null    object
8   class          891 non-null    category
9   who            891 non-null    object
10  adult_male     891 non-null    bool
11  deck          203 non-null    category
12  embark_town    889 non-null    object
13  alive          891 non-null    object
14  alone          891 non-null    bool
dtypes: bool(2), category(2), float64(2), int64(4), object(5)
memory usage: 80.7+ KB
None

```

Part A: Feature Construction

In [80]:

```

# Family Size
df['family_size'] = df['sibsp'] + df['parch'] + 1

# Fare Per Person
df['fare_per_person'] = df['fare'] / df['family_size']

df[['sibsp', 'parch', 'family_size', 'fare', 'fare_per_person']].head(10)

```

Out[80]:

	sibsp	parch	family_size	fare	fare_per_person
0	1	0	2	7.2500	3.62500
1	1	0	2	71.2833	35.64165
2	0	0	1	7.9250	7.92500
3	1	0	2	53.1000	26.55000
4	0	0	1	8.0500	8.05000
5	0	0	1	8.4583	8.45830
6	0	0	1	51.8625	51.86250
7	3	1	5	21.0750	4.21500
8	0	2	3	11.1333	3.71110
9	1	0	2	30.0708	15.03540

In [81]:

```

# Q. Give reasons as to why the new features might be important.

# A. family_size may be important because passengers travelling with family may have had
different survival behaviour compared with passengers travelling alone. Family members may
have helped each other during evacuation.
# fare_per_person may be useful because the original fare can represent the cost paid for
more than one passenger. Dividing the fare by family size can provide a better estimate of
the amount paid per passenger and may indirectly represent socioeconomic status.

```

Part B – Feature Selection and Feature Importance

In [82]:

```

#Q1
print(df.columns.tolist())

```

```

['survived', 'pclass', 'sex', 'age', 'sibsp', 'parch', 'fare', 'embarked', 'class', 'who',
 'adult_male', 'deck', 'embark_town', 'alive', 'alone', 'family_size', 'fare_per_person']

```

In [83]:

```

# Relevant Features

```

Relevant Features

```
#pclass
#sex
#age
#sibsp
#parch
#fare
#embarked
#family_size
#fare_per_person
```

In [84]:

```
# Irrelevant feature
# deck
```

In [85]:

```
# Redundant features
# class & pclass
# embark_town & embarked
# adult_male, who, alone also derived from sex, age, sibsp, and parch
```

In [86]:

```
columns_to_remove = [
    'class',
    'who',
    'adult_male',
    'deck',
    'embark_town',
    'alive',
    'alone'
]

df_selected = df.drop(columns=columns_to_remove)

print(df_selected.columns)
```

```
Index(['survived', 'pclass', 'sex', 'age', 'sibsp', 'parch', 'fare',
      'embarked', 'family_size', 'fare_per_person'],
      dtype='object')
```

In [87]:

```
# The relevant features include passenger class, sex, age, family information, fare, and
port of embarkation because these variables may affect survival. deck was removed due to
a large number of missing values. class and embark_town were removed because they duplica
te information contained in pclass and embarked. alive was removed because it directly re
presents the target variable and would cause data leakage. Other derived variables such a
s adult_male, who, and alone were removed to reduce redundancy.
```

In [88]:

```
# Q2
# Perform a chi-square test and/or construct a correlation matrix (heatmap)
df_corr = df_selected.copy()

# Convert sex into numeric values
df_corr['sex'] = df_corr['sex'].map({
    'male': 0,
    'female': 1
})

# Convert embarked into dummy variables
df_corr = pd.get_dummies(
    df_corr,
    columns=['embarked'],
    drop_first=True
)
```

```
# Fill missing numeric values
df_corr['age'] = df_corr['age'].fillna(df_corr['age'].median())

# Check resulting data
df_corr.head()
```

Out[88]:

	survived	pclass	sex	age	sibsp	parch	fare	family_size	fare_per_person	embarked_Q	embarked_S
0	0	3	0	22.0	1	0	7.2500	2	3.62500	False	True
1	1	1	1	38.0	1	0	71.2833	2	35.64165	False	False
2	1	3	1	26.0	0	0	7.9250	1	7.92500	False	True
3	1	1	1	35.0	1	0	53.1000	2	26.55000	False	True
4	0	3	0	35.0	0	0	8.0500	1	8.05000	False	True

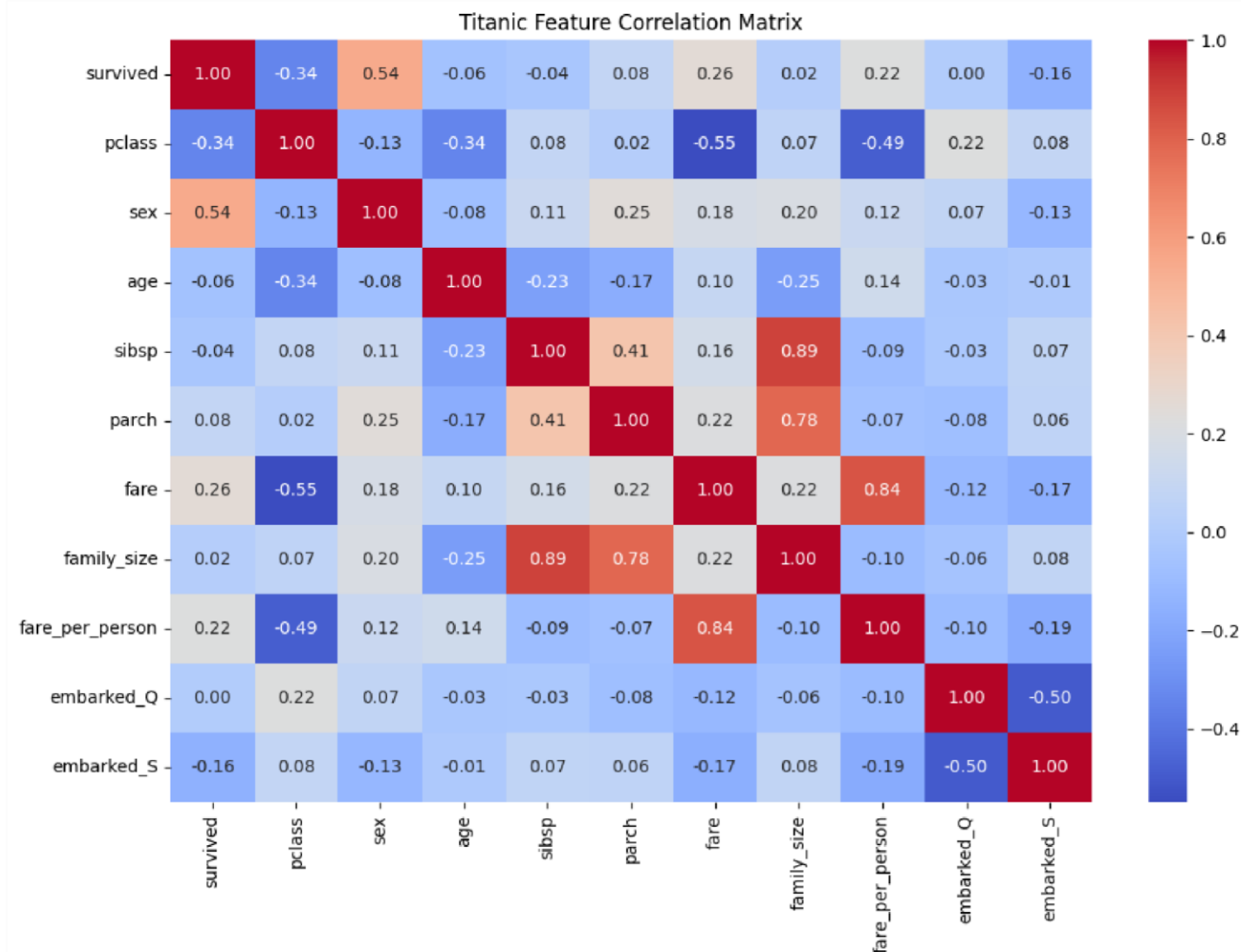
In [89]:

```
corr_matrix = df_corr.corr()

plt.figure(figsize=(12, 8))

sns.heatmap(
    corr_matrix,
    annot=True,
    cmap='coolwarm',
    fmt='.2f'
)

plt.title('Titanic Feature Correlation Matrix')
plt.show()
```



In [90]:

```
# Features that are highly correlated with the target variable
target_correlation = corr_matrix['survived'].sort_values(
    ascending=False
)

print(target_correlation)
```

```
survived      1.000000
sex           0.543351
fare          0.257307
fare_per_person 0.221600
parch         0.081629
family_size   0.016639
embarked_Q    0.003650
sibsp        -0.035322
age          -0.064910
embarked_S   -0.155660
pclass       -0.338481
Name: survived, dtype: float64
```

In [91]:

```
# looking for highly correlated features
print(
    corr_matrix['family_size']
    .sort_values(ascending=False)
)
```

```
family_size      1.000000
sibsp            0.890712
parch            0.783111
fare             0.217138
sex              0.200988
embarked_S       0.079977
pclass           0.065997
survived         0.016639
embarked_Q      -0.058592
fare_per_person -0.099173
age             -0.245619
Name: family_size, dtype: float64
```

In [92]:

```
# Q3
# Preparing X and Y
data = df_corr.copy()

X = data.drop('survived', axis=1)
y = data['survived']

print(X.columns)

Index(['pclass', 'sex', 'age', 'sibsp', 'parch', 'fare', 'family_size',
      'fare_per_person', 'embarked_Q', 'embarked_S'],
      dtype='object')
```

In [93]:

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

In [94]:

```
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
X_train_scaled = pd.DataFrame(
    X_train_scaled,
    columns=X.columns,
    index=X_train.index
)
```

```
X_test_scaled = pd.DataFrame(
    X_test_scaled,
    columns=X.columns,
    index=X_test.index
)
```

In [95]:

```
# Forward selection feature
logistic_model = LogisticRegression(
    max_iter=1000
)

sfs = SequentialFeatureSelector(
    logistic_model,
    n_features_to_select='auto',
    direction='forward',
    scoring='accuracy',
    cv=5
)

sfs.fit(X_train_scaled, y_train)
```

Out[95]:

- ▶ SequentialFeatureSelector
i ?
- ▶ estimator:
LogisticRegression
- ▶ LogisticRegression
?

In [96]:

```
selected_features = X.columns[
    sfs.get_support()
]

print("Selected Features:")
print(selected_features.tolist())
```

```
Selected Features:
['sex', 'age', 'sibsp', 'parch', 'family_size']
```

In [97]:

```
print(
    "Number of selected features:",
    len(selected_features)
)
```

Number of selected features: 5

In [98]:

```
# Train Logistic Regression using selected features
X_train_selected = X_train_scaled[
    selected_features
]
```

```

]

X_test_selected = X_test_scaled[
    selected_features
]

model = LogisticRegression(
    max_iter=1000
)

model.fit(
    X_train_selected,
    y_train
)

y_pred = model.predict(
    X_test_selected
)

accuracy = accuracy_score(
    y_test,
    y_pred
)

print(
    "Accuracy after feature selection:",
    accuracy
)

```

Accuracy after feature selection: 0.7877094972067039

In [99]:

```

# question wants the order too
remaining_features = list(X.columns)
chosen_features = []
selection_order = []

best_score = 0

while remaining_features:

    feature_scores = {}

    for feature in remaining_features:

        current_features = chosen_features + [feature]

        model = LogisticRegression(
            max_iter=1000
        )

        model.fit(
            X_train_scaled[current_features],
            y_train
        )

        predictions = model.predict(
            X_test_scaled[current_features]
        )

        score = accuracy_score(
            y_test,
            predictions
        )

        feature_scores[feature] = score

    best_feature = max(
        feature_scores,
        key=feature_scores.get
    )

```

```

current_score = feature_scores[
    best_feature
]

# Stop if adding features no longer improves accuracy
if current_score < best_score:
    break

chosen_features.append(best_feature)
remaining_features.remove(best_feature)

selection_order.append(
    (best_feature, current_score)
)

best_score = current_score

```

In [100]:

```

print("Forward Selection Order:")

for i, (feature, score) in enumerate(
    selection_order,
    start=1
):

    print(
        i,
        feature,
        "Accuracy:",
        round(score, 4)
    )

```

```

Forward Selection Order:
1 sex Accuracy: 0.7765
2 sibsp Accuracy: 0.7877
3 pclass Accuracy: 0.8045
4 fare Accuracy: 0.8101
5 fare_per_person Accuracy: 0.8101
6 parch Accuracy: 0.8101
7 age Accuracy: 0.8156
8 family_size Accuracy: 0.8156

```

In [101]:

```

print(
    "\nFinal Selected Features:",
    chosen_features
)

print(
    "Final Accuracy:",
    best_score
)

```

```

Final Selected Features: ['sex', 'sibsp', 'pclass', 'fare', 'fare_per_person', 'parch', '
age', 'family_size']
Final Accuracy: 0.8156424581005587

```

In [102]:

```

# How many features were selected
print(len(chosen_features))

```

8

In [103]:

```

# Forward Feature Selection selected 8 features from the original feature set.

```


In [104]:

```
# Forward Feature Selection selected X features from the original feature set.  
# No, all features were not necessarily selected. Forward Feature Selection only adds features that improve the predictive performance of the model. Features that provide little additional information or contain information already represented by another feature may not be selected.
```

In [105]:

```
# Compare selected features with correlation analysis and Random Forest feature importance.  
# Some of the features selected by Forward Feature Selection also showed strong relationships with survival in the correlation analysis and high importance in the Random Forest model. However, the methods may not select exactly the same features because correlation considers individual linear relationships, Random Forest captures nonlinear relationships, and Forward Selection evaluates combinations of features based on model performance.
```

In [106]:

```
# Q4  
# Train a Random Forest Classifier and display feature importance scores.  
  
rf = RandomForestClassifier(  
    n_estimators=200,  
    random_state=42  
)  
  
rf.fit(  
    X_train,  
    y_train  
)
```

Out[106]:

```
▼                                RandomForestClassifier                                i ?  
  
RandomForestClassifier(n_estimators=200, random_state=42)
```

In [107]:

```
feature_importance = pd.DataFrame({  
    'Feature': X.columns,  
    'Importance': rf.feature_importances_  
})  
  
feature_importance = feature_importance.sort_values(  
    by='Importance',  
    ascending=False  
)  
  
print(feature_importance)
```

	Feature	Importance
1	sex	0.252286
2	age	0.217735
5	fare	0.177048
7	fare_per_person	0.164113
0	pclass	0.066400
6	family_size	0.047478
3	sibsp	0.023049
9	embarked_S	0.022636
4	parch	0.020401
8	embarked_Q	0.008854

In [108]:

```
plt.figure(figsize=(10, 6))  
  
sns.barplot(  
    data=feature_importance,
```

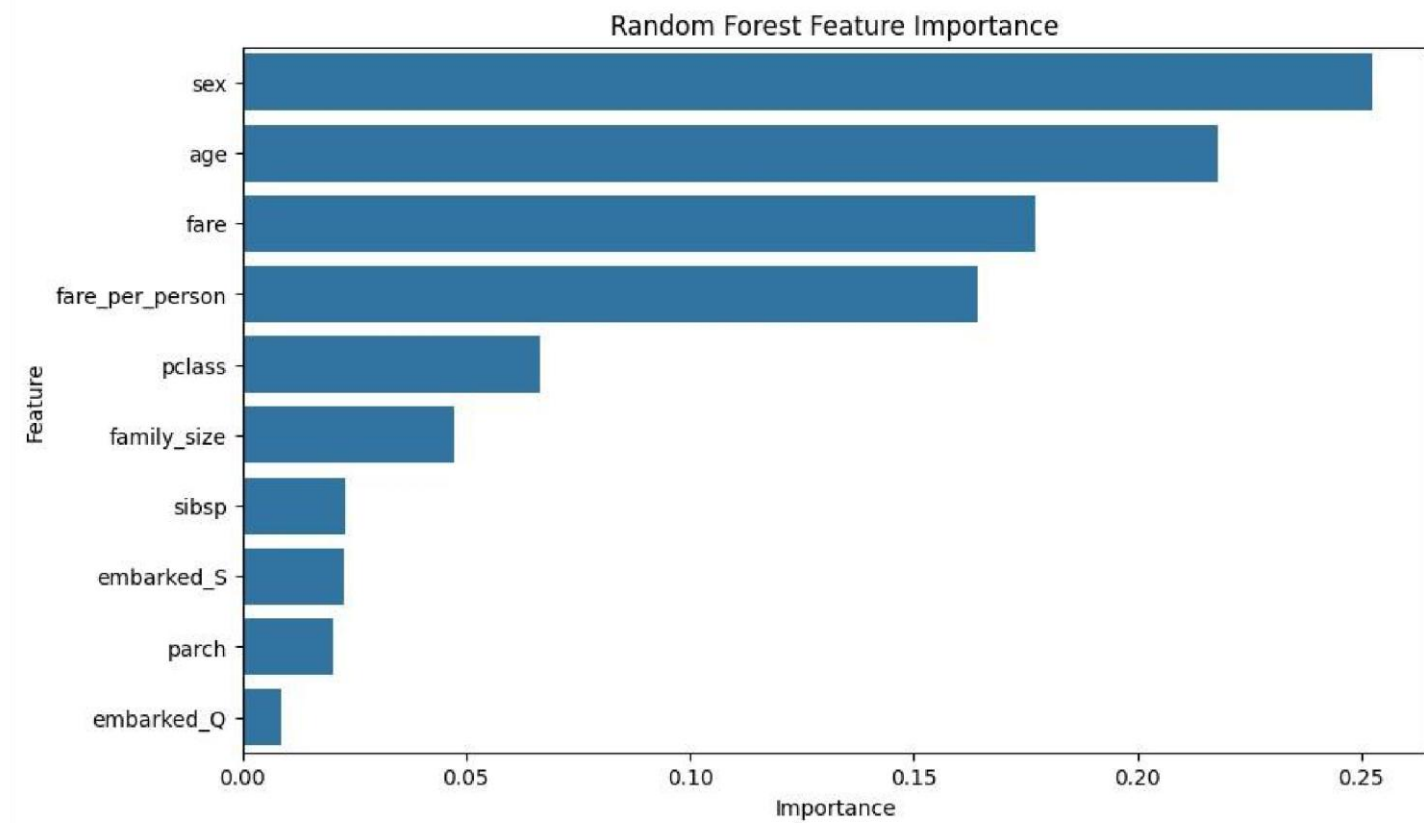
```

        x='Importance',
        y='Feature'
    )

plt.title(
    'Random Forest Feature Importance'
)

plt.show()

```



In [109]:

```

# highest imporrant feature
print(
    "Highest Importance:"
)

print(
    feature_importance.iloc[0]
)

```

```

Highest Importance:
Feature          sex
Importance    0.252286
Name: 1, dtype: object

```

In [110]:

```

# lowest important feature
print(
    "Lowest Importance:"
)

print(
    feature_importance.iloc[-1]
)

```

```

Lowest Importance:
Feature    embarked_Q
Importance    0.008854
Name: 8, dtype: object

```

In [111]:

Compare with correlation
The results may be similar but are not expected to be identical. Correlation measures mainly linear relationships between individual variables and the target. Random Forest feature importance measures how useful a variable is when making decision-tree splits and can identify nonlinear relationships and interactions between variables. Therefore, a feature with low correlation can still obtain relatively high Random Forest importance.

In [112]:

```
# Q5
# Final selected features
final_features = [
    'pclass',
    'sex',
    'age',
    'fare',
    'family_size',
    'fare_per_person',
    'embarked_Q',
    'embarked_S'
]

print(final_features)
```

```
['pclass', 'sex', 'age', 'fare', 'family_size', 'fare_per_person', 'embarked_Q', 'embarked_S']
```

In [113]:

The final feature set was selected by comparing results from manual domain-based selection, correlation analysis, Random Forest feature importance and Forward Feature Selection. Features such as sex, pclass, age, and fare were retained because they showed useful predictive information. family_size was retained because it summarizes family relationships, allowing sibsp and parch to be removed as potentially redundant variables. Features that duplicated other information or contributed little predictive value were excluded. Combining several feature-selection methods provides a more reliable feature set than depending on only one method.

Part C – Principal Component Analysis (PCA)

In [114]:

```
# Creating a numerical dataset
pca_features = [
    'pclass',
    'age',
    'sibsp',
    'parch',
    'fare',
    'family_size',
    'fare_per_person'
]

X_pca = df[pca_features].copy()

X_pca['age'] = X_pca['age'].fillna(
    X_pca['age'].median()
)

print(X_pca.head())
```

	pclass	age	sibsp	parch	fare	family_size	fare_per_person
0	3	22.0	1	0	7.2500	2	3.62500
1	1	38.0	1	0	71.2833	2	35.64165
2	3	26.0	0	0	7.9250	1	7.92500
3	1	35.0	1	0	53.1000	2	26.55000
4	3	35.0	0	0	8.0500	1	8.05000

In [115]:

```
# Q1
```

```
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(  
    X_pca  
)
```

```
print(X_scaled[:5])
```

```
[[ 0.82737724 -0.56573646  0.43279337 -0.47367361 -0.50244517  0.05915988  
 -0.4547978 ]  
 [-1.56610693  0.66386103  0.43279337 -0.47367361  0.78684529  0.05915988  
  0.43899428]  
 [ 0.82737724 -0.25833709 -0.4745452  -0.47367361 -0.48885426 -0.56097483  
 -0.33475695]  
 [-1.56610693  0.4333115  0.43279337 -0.47367361  0.42073024  0.05915988  
  0.18518744]  
 [ 0.82737724  0.4333115  -0.4745452  -0.47367361 -0.48633742 -0.56097483  
 -0.33126739]]
```

In [116]:

```
X_scaled_df = pd.DataFrame(  
    X_scaled,  
    columns=pca_features  
)
```

```
X_scaled_df.head()
```

Out[116]:

	pclass	age	sibsp	parch	fare	family_size	fare_per_person
0	0.827377	-0.565736	0.432793	-0.473674	-0.502445	0.059160	-0.454798
1	-1.566107	0.663861	0.432793	-0.473674	0.786845	0.059160	0.438994
2	0.827377	-0.258337	-0.474545	-0.473674	-0.488854	-0.560975	-0.334757
3	-1.566107	0.433312	0.432793	-0.473674	0.420730	0.059160	0.185187
4	0.827377	0.433312	-0.474545	-0.473674	-0.486337	-0.560975	-0.331267

In [117]:

```
# Feature scaling is important before PCA because PCA is based on variance. Features with  
larger numerical scales naturally have larger variances and may dominate the principal co  
mponents. Standardization transforms the features to approximately zero mean and unit var  
iance, allowing each feature to contribute more fairly to PCA.
```

In [118]:

```
# Q2  
pca = PCA()  
  
X_pca_transformed = pca.fit_transform(  
    X_scaled  
)
```

In [119]:

```
print(  
    "Principal Components:"  
)  
  
print(  
    pca.components_  
)
```

Principal Components:

```
[[ 1.07142435e-01 -2.54113835e-01  5.35498205e-01  4.92880356e-01  
  1.03543321e-01  6.12229489e-01 -9.33333645e-02]  
 [-5.01127091e-01  2.04620583e-01  3.23636691e-02  8.19497329e-02  
  6.06334061e-01  6.30601871e-02  5.72382136e-01]]
```

```

6.06334061e-01  6.30601871e-02  5.72382136e-01]
[-2.59422772e-01  8.37896304e-01  8.95654997e-02  1.81747678e-01
-2.14969867e-01  1.52013075e-01 -3.46778593e-01]
[-1.01578341e-02 -9.79513119e-02 -6.54385221e-01  7.44117561e-01
 1.38677905e-02 -7.55009615e-02 -4.97864051e-02]
[ 8.10360394e-01  4.26460742e-01 -2.20663752e-02  6.94961754e-02
 1.88036164e-01  1.96375320e-02  3.46968864e-01]
[ 1.15346491e-01  4.80991510e-03 -5.79676072e-02 -1.15738320e-01
 7.34762430e-01 -9.74398182e-02 -6.48495260e-01]
[ 1.91372387e-17 -1.21802961e-16 -5.21636943e-01 -3.81293792e-01
-9.97407284e-16  7.63223391e-01  1.12443949e-15]]

```

In [120]:

```

print(
    "Explained Variance Ratio:"
)

print(
    pca.explained_variance_ratio_
)

```

Explained Variance Ratio:
[0.36783815 0.33443073 0.13054036 0.08383248 0.06954903 0.01380925
0.]

In [121]:

```

cumulative_variance = np.cumsum(
    pca.explained_variance_ratio_
)

print(
    "Cumulative Explained Variance:"
)

print(
    cumulative_variance
)

```

Cumulative Explained Variance:
[0.36783815 0.70226888 0.83280924 0.91664172 0.98619075 1.
1.]

In [122]:

```

pca_results = pd.DataFrame({
    'Principal Component':
        range(
            1,
            len(pca.explained_variance_ratio_) + 1
        ),
    'Explained Variance':
        pca.explained_variance_ratio_,
    'Cumulative Variance':
        cumulative_variance
})

print(pca_results)

```

	Principal Component	Explained Variance	Cumulative Variance
0	1	0.367838	0.367838
1	2	0.334431	0.702269
2	3	0.130540	0.832809
3	4	0.083832	0.916642
4	5	0.069549	0.986191
5	6	0.013809	1.000000
6	7	0.000000	1.000000

In [123]:

```
# Q3

plt.figure(figsize=(8, 5))

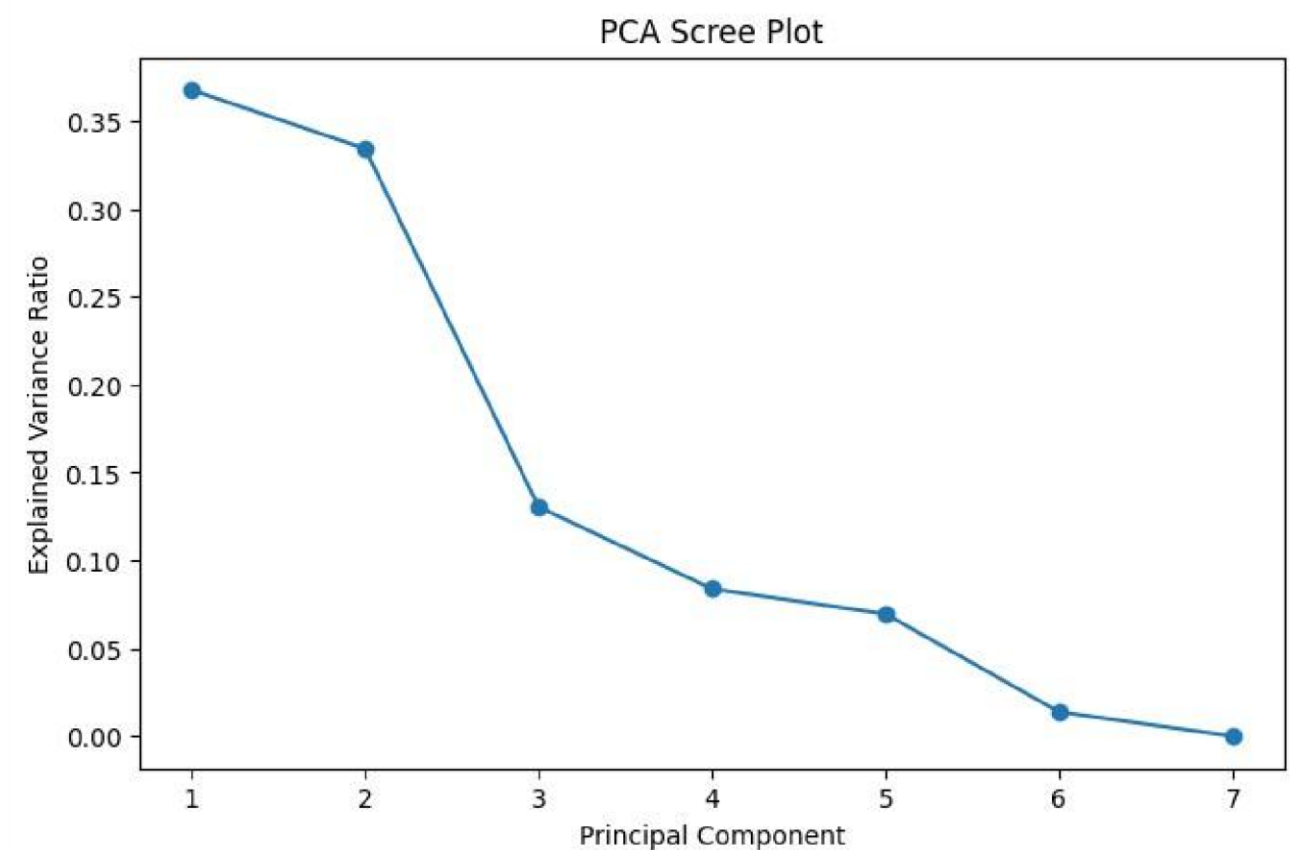
plt.plot(
    range(
        1,
        len(pca.explained_variance_ratio_) + 1
    ),
    pca.explained_variance_ratio_,
    marker='o'
)

plt.xlabel(
    'Principal Component'
)

plt.ylabel(
    'Explained Variance Ratio'
)

plt.title(
    'PCA Scree Plot'
)

plt.show()
```



In [124]:

```
plt.figure(figsize=(8, 5))

plt.plot(
    range(
        1,
        len(cumulative_variance) + 1
    ),
    cumulative_variance,
    marker='o'
)

plt.axhline(
    y=0.95,
```

```

        linestyle='--'
    )

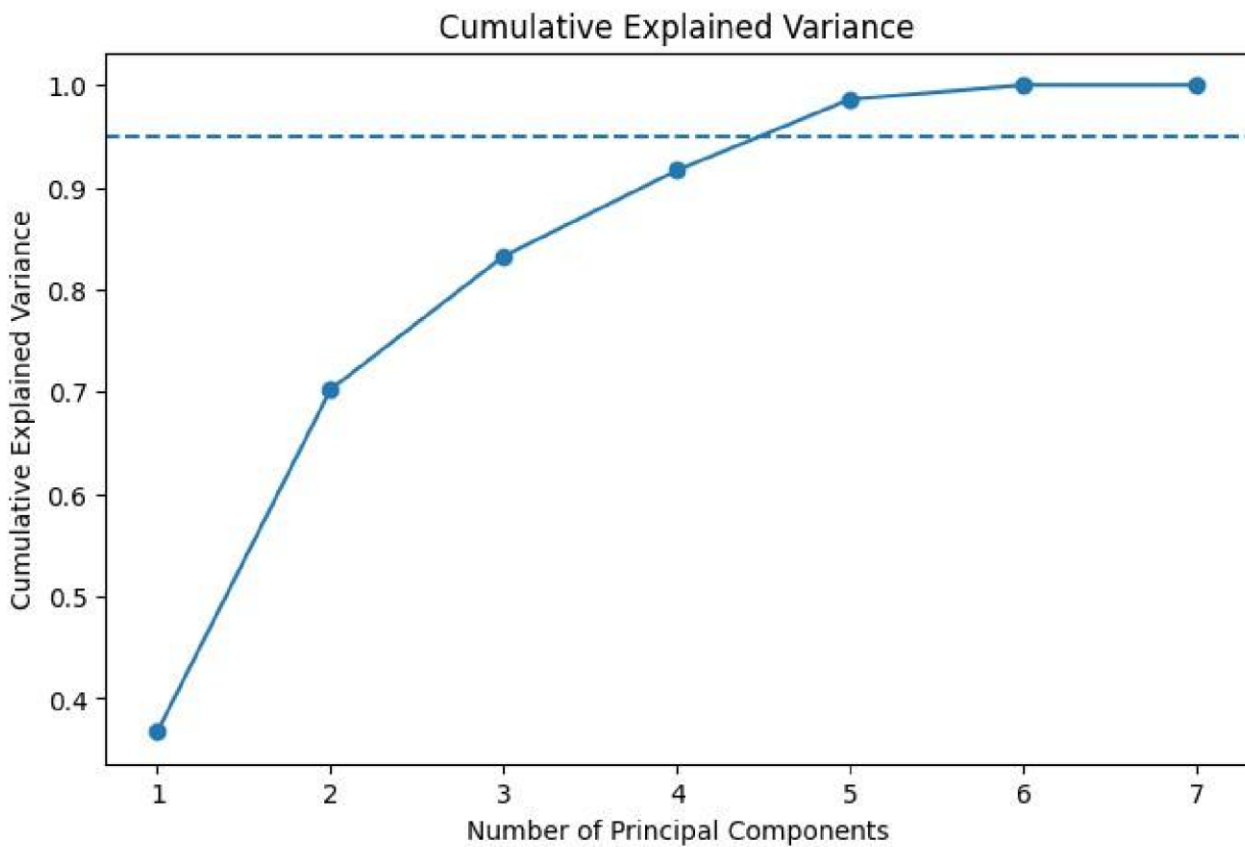
plt.xlabel(
    'Number of Principal Components'
)

plt.ylabel(
    'Cumulative Explained Variance'
)

plt.title(
    'Cumulative Explained Variance'
)

plt.show()

```



In [125]:

```

# Q4
n_components_95 = np.argmax(
    cumulative_variance >= 0.95
) + 1

print(
    "Number of components required for 95% variance:",
    n_components_95
)

```

Number of components required for 95% variance: 5

In [126]:

```

original_features = X_pca.shape[1]

print(
    "Original numerical features:",
    original_features
)

```

Original numerical features: 7

In [127]:


```
print(
    "Principal components required:",
    n_components_95
)
```

Principal components required: 5

In [128]:

```
reduction = (
    original_features -
    n_components_95
)

print(
    "Dimensionality reduced by:",
    reduction
)
```

Dimensionality reduced by: 2

In [129]:

The PCA analysis originally used 7 numerical variables. To preserve at least 95% of the total variance, X principal components were required. Therefore, dimensionality was reduced from 7 dimensions to X dimensions, resulting in a reduction of 7 - X dimensions.

In [130]:

```
# Q5
pca_95 = PCA(
    n_components=n_components_95
)

X_reduced = pca_95.fit_transform(
    X_scaled
)

print(
    "Shape before PCA:",
    X_scaled.shape
)

print(
    "Shape after PCA:",
    X_reduced.shape
)
```

Shape before PCA: (891, 7)

Shape after PCA: (891, 5)

Part D – Class Imbalance and Data Sampling

In [131]:

```
class_counts = df['survived'].value_counts()

print(class_counts)
```

```
survived
0      549
1      342
Name: count, dtype: int64
```

In [132]:

```
class_percentages = (
    df['survived']
    .value_counts(normalize=True)
    * 100
)
```

```
)  
  
print(class_percentages)
```

```
survived  
0    61.616162  
1    38.383838  
Name: proportion, dtype: float64
```

In [133]:

```
class_percentages = (  
    df['survived']  
    .value_counts(normalize=True)  
    * 100  
)  
  
print(class_percentages)
```

```
survived  
0    61.616162  
1    38.383838  
Name: proportion, dtype: float64
```

In [134]:

```
# The Titanic dataset is moderately imbalanced. Approximately 61.6% of passengers belong  
to class 0 (did not survive), while approximately 38.4% belong to class 1 (survived). The  
imbalance is not extremely severe, but one class contains noticeably more observations th  
an the other.
```

In [135]:

```
# Q2  
X = df_corr.drop(  
    'survived',  
    axis=1  
)  
  
y = df_corr['survived']
```

In [136]:

```
X_train_random, X_test_random, y_train_random, y_test_random = train_test_split(  
    X,  
    y,  
    test_size=0.2,  
    random_state=42  
)
```

In [137]:

```
print("Random Training Distribution:")  
  
print(  
    y_train_random.value_counts(  
        normalize=True  
    )  
)  
  
print("\nRandom Testing Distribution:")  
  
print(  
    y_test_random.value_counts(  
        normalize=True  
    )  
)
```

```
Random Training Distribution:  
survived  
0    0.623596  
1    0.376404
```

Name: proportion, dtype: float64

Random Testing Distribution:

survived

0 0.586592

1 0.413408

Name: proportion, dtype: float64

In [138]:

```
X_train_strat, X_test_strat, y_train_strat, y_test_strat = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

In [139]:

```
print(
    "Stratified Training Distribution:"
)

print(
    y_train_strat.value_counts(
        normalize=True
    )
)

print(
    "\nStratified Testing Distribution:"
)

print(
    y_test_strat.value_counts(
        normalize=True
    )
)
```

Stratified Training Distribution:

survived

0 0.616573

1 0.383427

Name: proportion, dtype: float64

Stratified Testing Distribution:

survived

0 0.614525

1 0.385475

Name: proportion, dtype: float64

In [140]:

```
# Which splitting method better preserves the original class distribution?

# Stratified train-test splitting better preserves the original class distribution because it ensures that approximately the same proportion of each target class appears in both the training and testing datasets.
```

In [141]:

```
# Why is stratified splitting preferred for classification problems?
# Stratified splitting is preferred because it prevents one subset from accidentally containing too many or too few samples from a particular class. This is especially important for imbalanced classification problems because both training and testing datasets should properly represent each class.
```

In [142]:

```
# Q3
```



```
# before oversampling
print(
    "Before Oversampling:"
)

print(
    y_train_strat.value_counts()
)
```

```
Before Oversampling:
survived
0      439
1      273
Name: count, dtype: int64
```

In [143]:

```
# Random oversampling
ros = RandomOverSampler(
    random_state=42
)

X_train_over, y_train_over = ros.fit_resample(
    X_train_strat,
    y_train_strat
)
```

In [144]:

```
print(
    "After Oversampling:"
)

print(
    y_train_over.value_counts()
)
```

```
After Oversampling:
survived
1      439
0      439
Name: count, dtype: int64
```

In [145]:

```
# compare before and after
print(
    "Before:",
    X_train_strat.shape
)

print(
    "After:",
    X_train_over.shape
)
```

```
Before: (712, 10)
After: (878, 10)
```

In [146]:

```
# What changes do you observe after oversampling?

# Before oversampling, class 0 had more observations than class 1. Random Oversampling in-
creased the number of observations belonging to the minority class by duplicating existin-
g minority-class samples. After oversampling, both classes contained the same number of o-
bservations, and the total training dataset became larger.
```

In [147]:

```
# One advantage and disadvantage?
```

Advantage

It prevents the model from being strongly biased toward the majority class.

Disadvantage

Because existing minority-class examples are duplicated, the model may memorize those examples, increasing the risk of overfitting.

In [148]:

```
# Q4
# Before undersampling

print(
    "Before Undersampling:"
)

print(
    y_train_strat.value_counts()
)
```

```
Before Undersampling:
survived
0      439
1      273
Name: count, dtype: int64
```

In [149]:

```
# apply undersampling
rus = RandomUnderSampler(
    random_state=42
)

X_train_under, y_train_under = rus.fit_resample(
    X_train_strat,
    y_train_strat
)
```

In [150]:

```
print(
    "After Undersampling:"
)

print(
    y_train_under.value_counts()
)
```

```
After Undersampling:
survived
0      273
1      273
Name: count, dtype: int64
```

In [151]:

```
print(
    "Before:",
    X_train_strat.shape
)
print(
    "After:",
    X_train_under.shape
)
```

```
Before: (712, 10)
After: (546, 10)
```


In [152]:

```
# What changes do you observe?  
# Random Undersampling reduced the number of majority-class observations until it matched  
the minority class. As a result, the classes became balanced, but the overall size of the  
training dataset decreased.
```

In [153]:

```
#Advantage  
#Training becomes faster because fewer observations are used and class imbalance is removed.  
  
#Disadvantage  
#Some majority-class observations are discarded, which may result in the loss of useful information.
```

In [154]:

```
# Q5  
print("Original Training Dataset")  
print(y_train_strat.value_counts())  
print("Size:", len(y_train_strat))  
  
print("\nRandom Oversampling")  
print(y_train_over.value_counts())  
print("Size:", len(y_train_over))  
  
print("\nRandom Undersampling")  
print(y_train_under.value_counts())  
print("Size:", len(y_train_under))
```

```
Original Training Dataset  
survived  
0      439  
1      273  
Name: count, dtype: int64  
Size: 712
```

```
Random Oversampling  
survived  
1      439  
0      439  
Name: count, dtype: int64  
Size: 878
```

```
Random Undersampling  
survived  
0      273  
1      273  
Name: count, dtype: int64  
Size: 546
```

In [155]:

```
# Which technique increases the size of the dataset?  
# Random Oversampling.
```

In [156]:

```
# Which technique decreases the size of the dataset?  
# Random Undersampling.
```

In [157]:

```
# Which technique may lead to overfitting?  
# Random Oversampling because it duplicates existing minority-class samples.
```

In [158]:

```
# Which technique may result in loss of useful information?
```

```
# Random Undersampling because majority-class observations are removed.
```

In [159]:

```
# Which technique would you recommend?  
# For the Titanic dataset, Random Oversampling is generally preferable to Random Undersam  
pling because the dataset contains only about 891 observations. Undersampling would disca  
rd useful passenger information and further reduce the amount of training data. However,  
SMOTE may be a better option than simple Random Oversampling because it generates synthe  
tic minority-class observations instead of repeatedly duplicating existing observations.
```

In [160]:

```
# Q6  
#Before SMOTE  
print(  
    "Before SMOTE:"  
)  
print(  
    y_train_strat.value_counts()  
)
```

```
Before SMOTE:  
survived  
0      439  
1      273  
Name: count, dtype: int64
```

In [161]:

```
# Apply SMOTE  
smote = SMOTE(  
    random_state=42  
)  
X_train_smote, y_train_smote = smote.fit_resample(  
    X_train_strat,  
    y_train_strat  
)
```

In [162]:

```
smote = SMOTE(  
    random_state=42  
)  
X_train_smote, y_train_smote = smote.fit_resample(  
    X_train_strat,  
    y_train_strat  
)
```

In [163]:

```
print(  
    "Before SMOTE:",  
    X_train_strat.shape  
)  
print(  
    "After SMOTE:",  
    X_train_smote.shape  
)
```

```
Before SMOTE: (712, 10)  
After SMOTE: (878, 10)
```

In [164]:

```
# How is SMOTE different from Random Oversampling?  
# Random Oversampling balances the classes by randomly duplicating existing minority-clas  
s observations. SMOTE instead creates new synthetic minority-class samples by interpolati  
ng between neighbouring minority observations.
```

In [166]:

In [166]:

```
# Why is SMOTE generally preferred over simply duplicating samples?
# SMOTE is generally preferred because it introduces new synthetic samples rather
than repeatedly copying the same observations. This provides greater diversity in
the minority class and can reduce the risk of the model memorizing duplicated
observations. However, SMOTE should still be used carefully because synthetic
observations may sometimes introduce noise.
```